

E-Voting App Report

System Structure and Design Decisions

1. System Structure

The E-Voting app is organized using a **modular structure**, where the application is divided into multiple components that each perform a specific role. This structure improves code organization, readability, and maintainability.

The project structure is shown below:

```
E-Voting App
├── models
├── services
├── storage
├── ui
├── utils
├── data
├── main.py
└── evoting_data.json
```

Each module in the system is responsible for a specific function.

The **models module** defines the core entities used by the system. These entities represent real-world components of the election system such as voters, candidates, administrators, polls, positions, voting stations, and votes. The models provide structured representations of the data used throughout the application.

The **services module** contains the main business logic of the system. This module processes operations such as authentication, voter verification, poll creation, candidate management, vote casting, and result computation. By separating business logic into services, the system ensures that application rules are applied consistently.

The **storage module** manages data persistence and system state. It is responsible for loading stored data when the system starts and saving updated data when changes occur. Instead of using a database, the system stores information in a JSON file named **evoting_data.json**, which contains the system's data.

The **ui module** handles all user interaction through the command-line interface. It provides menus, prompts, dashboards, and formatted tables that allow administrators and voters to interact with the system.

The **utils module** provides shared helper functions used across the application. These include password hashing, logging utilities, timestamp generation, voter card creation, and terminal display helpers.

Finally, the **main.py file** acts as the central controller of the application. It connects all modules together and manages the overall program flow, including login processes, dashboard navigation, and execution of service functions.

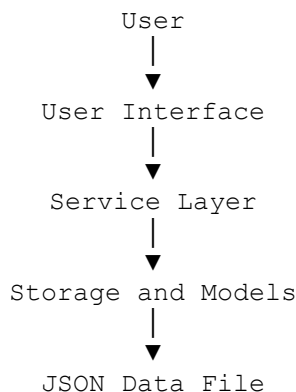
2. Design Decisions

Several design decisions were made to ensure that the system is well structured and maintainable.

One important decision was the use of **modular design**. The system was divided into independent modules such as models, services, storage, UI, and utilities. This makes the code easier to understand and maintain because each module performs a specific role.

Another key design decision was the use of **separation of concerns**. Each module is responsible for a single type of functionality. For example, the UI module focuses only on displaying menus and collecting input, while the services module handles system logic such as voting and poll management. This separation reduces complexity and improves the clarity of the system.

The system also follows a **layered architecture**, which organizes the interaction between components. In this architecture, the user interface communicates with the service layer, which then interacts with the storage and model layers.



This structure ensures that system rules are validated in the service layer before data is stored or modified.

Another design decision was the use of **JSON file storage** instead of a traditional database. JSON was selected because it is lightweight, easy to implement, and suitable for a console-based application. It allows the system to store structured data without requiring additional database setup. Overall, these design decisions help ensure that the system remains organized, maintainable, and easy to extend.